Experiment6 Report

R.VIKAS RAJ, EE19B108, Batch 1

Objectives

To understand ARM assembly, the architecture and the instruction set by making three programs and emulating them in MicroVision.

Software Used

- Keil Microvision with legacy support installed

Architecture

- ARM is a 32-bit processor. This means the word size is 32 bits.
- The ARM processor has a large number of registers.
- Out of the 37 registers, 30 are general purpose registers, 6 are status registers and 1 is the program counter.
- ARM has a RISC instruction set.
- It has limited addressing modes and has a load and store architecture.
- It has a fixed instruction length of 32-bit

Factorial

Problem

Compute the factorial of a given number using an ARM processor through assembly programming

Code

```
    AREA program, CODE, READONLY
    ENTRY
main
    LDR R0, NUM1
    MOV R1, #1

loop
    CMP R0, #1
     BEQ end_loop
     MUL R3, R1, R0    ; Use R3 to store the result of R1 * R0
     MOV R1, R3        ; Move the result back into R1
    SUB R0, R0, #1
B loop

end_loop
     STR R1, result
     SWI &11

NUM1 DCW &0004
```

```
result DCW &0000
    END
```


Working :

- The first line tells the compiler that the given area is CODE. The READ ONLY says that the code under is only for reading. This is not required since the program is stored in ROM but it is a good practice to do that.
- ENTRY marks the beginning of the program.
- DCW in the last few lines stands for define constant word. This is used to store a constant word of 16 bytes in memory.
- The first step is to load the Number into registers. The LDR instruction loads the word pointed by the label NUM1. I am loading the number into two registers R0 and R3.
- One is used to keep track of the next number to be multiplied to get the factorial and the second one is used to keep track of the cumulative product so far.
- Then I am using the register R1 to move the cumulative product after each MUL operation. This is because multiplying the contents in a register by contents in another register and storing in the same register is not supported.
- The program then enters into a loop which continually decrements the contents of R0 till it becomes 1 and multiplies the number with the cumulative product.
- After the factorial calculation is complete, the program enters the end loop label. Here the result is stored.
- END marks the end of the program.

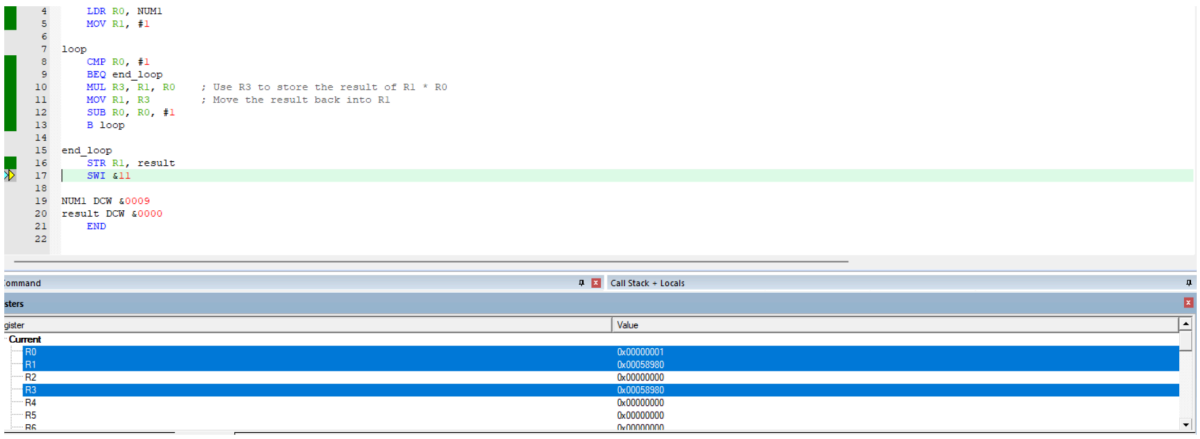

FIGURE 1 : SHOWS THE FACTORIAL OF 9 STORED AT R3

Word Manupulation

Problem

Combine the low four bits of each of the four consecutive bytes beginning at LIST into one 16-bit halfword. Call it as A halfword. Similarly, combine the higher four bits of each of the four consecutive bytes beginning at LIST into one 16-bit halfword, which is called as 'B' halfword. Combine BA (in that order) and store the result in the 32-bit variable RESULT. (Meaning store B in the higher 16-bit and A in the lower 16-bit of RESULT).

Code

```
        AREA  program , CODE , READONLY
        ENTRY
main
        LDR  R0 , = LIST
        LDRB  R1 , [ R0 ]
        LDRB  R2 , [ R0 , #1]
        LDRB  R3 , [ R0 , #2]
        LDRB  R4 , [ R0 , #3]

        AND  R5 , R1 , #0 x0F
        LSL  R5 , R5 , #4
        AND  R6 , R2 , #0 x0F
        ORR  R5 , R5 , R6
        LSL  R5 , R5 , #4
        AND  R6 , R3 , #0 x0F
        ORR  R5 , R5 , R6
        LSL  R5 , R5 , #4
        AND  R6 , R4 , #0 x0F
        ORR  R5 , R5 , R6
        MOV  R7 , R5

        MOV  R5 , R1 , LSR #4
        LSL  R5 , R5 , #4
        MOV   R6 , R2 , LSR #4
        ORR  R5 , R5 , R6
        LSL  R5 , R5 , #4
        MOV  R6 , R3 , LSR #4
        ORR  R5 , R5 , R6
        LSL  R5 , R5 , #4
        MOV  R6 , R4 , LSR #4
        ORR  R5 , R5 , R6
        MOV  R8 , R5

       LSL  R8 , R8 , #16
       ORR   R8 , R8 , R7


        STR   R8 , RESULT

        SWI  &11

LIST    DCB   0 x15 , 0 x34 , 0 x66 , 0 x74
RESULT   DCD   0 x00000000

        END
```

Working :

- The beginning and the ending of the program is the same as the first program.
- The four bytes are stored using the directive DCB in four consecutive locations.
- The LDR in main in loading the memory address of LIST into R0.
- The next four LDRB are loading the consecutive bytes into the registers R1, R2, R3 and R4.
- After this is done the two halves of the bytes are separated by anding with the corresponding number in binary. The lower four nibbles and upper four nubbles are combined as required by using a series

of Left shifts and or operations.

• Finally the number is being stored in the address marked by RESULT.

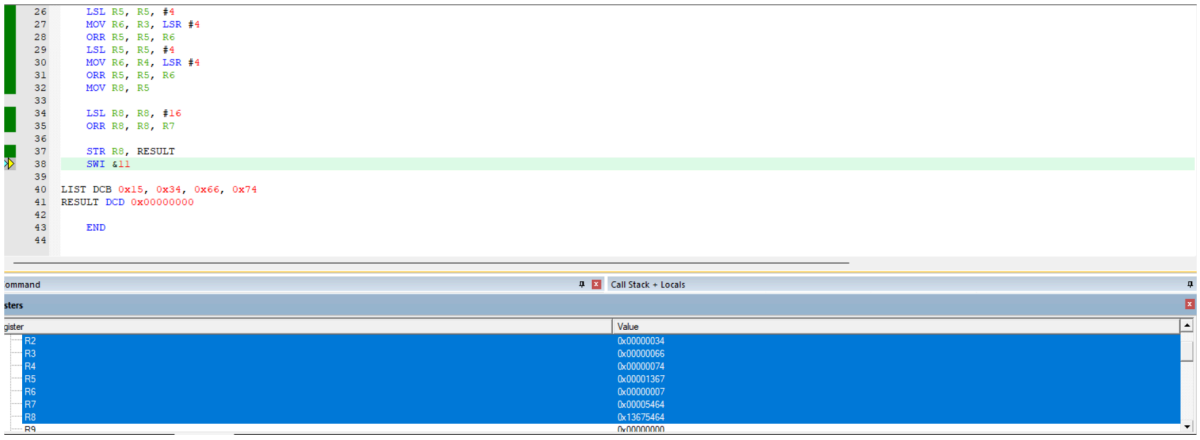

FIGURE 2 : SHOWS MULTIPLICATION OF TWO 16-BIT NUMBERS

Odd or Even

Problem

Given a 32 bit number, identify whether it is an even or odd. (You implementation should not involve division).

Code

```
        AREA  program , CODE , READONLY
        ENTRY
main
        LDR R0 , NUM
        AND R0 , R0 , #1
        CMP R0 , #1
        BEQ odd
        LDR R1 , =0
        B end_of_program
odd
        LDR R1 , =1
end_of_program
        SWI &11
NUM DCD &1237
        END
```

Working :

• The beginning and ending still remain the same. The label NUM is used to point to the input number. • The program first loads this number into R0 using the LDR instruction.

• Then the number is anded with 1.

• This number is then compared with the number 1.

- If the number is odd, the comparison will result in an equal flag, and the program jumps to the odd label, where R1 is loaded with 1 indicating the number is odd.
- If the number is even R1 is loaded with 0.

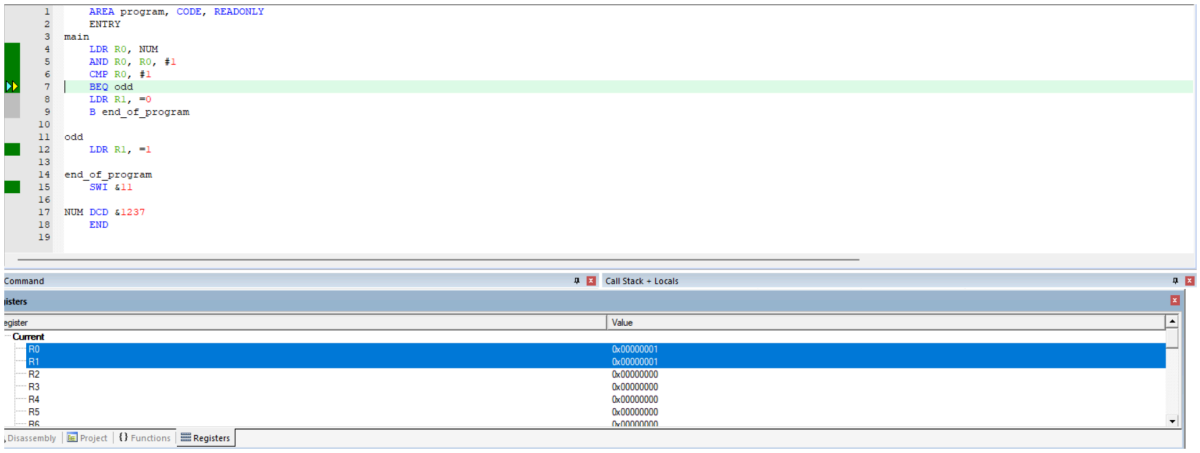

FIGURE 3 : SHOWS THE NUMBER 1287 IS ODD AS R1 IS 1

Inferences and conclusion

Similar to AVR, ARM is also a RISC-based processor. It used a load and store architecture. It has many general-purpose registers which can be used to run a wide range of tasks. Being a 32-bit architecture, ARM-based microcontrollers can operate on a wider range of numbers. ARM-based processors typically have a higher clock speed and better performance than AVR-based microcontrollers. This makes it useful for real-time applications.

The ARM instruction set is more complex than AVR. All the instructions are of 32 bits. ARM micro controllers generally have larger memory capacities compared to AVR. The 32-bit architecture allows ARM processors to handle a larger addressable memory space. ARM supports load/store multiple instructions, which can load or store multiple registers to/from memory in a single instruction.

ARM follows the Harvard architecture, similar to AVR, where program and data memories are separate. This allows simultaneous access to both instruction and data memory, improving throughput and efficiency.